

Generative Design of High-Strength Steel Beams Using Reinforcement Learning with Manufacturability-Aware Optimization

Author: Muhammad Shifa

Affiliation: Shaheed Abad, Khyber Pakhtunkhwa, Pakistan

Correspondence: [Contact information]

Abstract

This paper presents a comprehensive reinforcement learning (RL) framework for generative design of high-strength steel (HSS) beams with integrated manufacturability constraints. We develop a manufacturability-aware cost model that enforces physical feasibility rules derived from published section catalogs and material standards: only geometrically admissible sections receive economical rolled fabrication cost factors. Against a genetic algorithm (GA) baseline achieving 1.47% mean cost gap over 142 span-load contexts, our Proximal Policy Optimization (PPO) agent trained with feasibility-gated rewards reaches 6.34% mean gap while using 43× fewer structural evaluations at inference. We demonstrate that a reparameterized design space with feasibility-by-construction enables blind uniform sampling to achieve 5.9% mean gap at matched budget, indicating that action space formulation—not algorithm choice—is the primary bottleneck. A leave-one-out k-nearest-neighbor predictor achieves 2.00% mean / 0.64% median gap with no training. The framework, complete experimental results spanning five experiment phases (E0–E5), and open-source implementation support reproducible research in machine learning for structural engineering.

Keywords: Reinforcement learning, generative design, high-strength steel, manufacturability constraints, structural optimization, Proximal Policy Optimization

1. Introduction

1.1 Background and Motivation

The integration of machine learning methods into structural engineering design workflows has attracted significant research interest, driven by the promise of amortized inference cost and interactive design exploration [1–3]. Among machine learning paradigms, reinforcement learning (RL) offers particular advantages for generative design: policies trained offline can generate structurally sound designs at inference time orders of magnitude faster than direct optimization methods such as genetic algorithms or gradient-based optimizers [4].

High-strength steel (HSS) beams represent a compelling application domain. Grades S460–S690 provide 30–90% higher yield strength than conventional S355 steel, enabling lighter cross-sections, reduced self-weight, and lower embodied carbon [5]. However, HSS design introduces

complexity: higher grades are available only as plate products (EN 10025-6), meaning beams in S500–S690 must be fabricated as welded plate girders rather than hot-rolled sections [6]. This distinction carries significant cost implications, as welded fabrication incurs approximately $2.8\times$ higher fabrication factors than rolled sections [7].

1.2 Research Gap

Existing RL approaches to structural design have treated section type (rolled vs. welded) as a free decision variable decoupled from geometry and material grade [8,9]. This formulation permits designs to claim economical rolled fabrication cost factors despite employing plate-girder proportions and high-strength grades—a physical impossibility. Without manufacturability constraints, reported performance estimates reflect an idealized design space that cannot be realized in practice.

Furthermore, published comparisons between RL policies and direct optimization methods report widely varying results: some claim RL matches or exceeds genetic algorithm quality at $10\text{--}100\times$ lower inference cost [10], while others report RL trailing by 5–15 percentage points in cost gap [11]. We hypothesize that inconsistent cost modeling and absence of manufacturability constraints contribute to this variance.

1.3 Objectives and Contributions

This work develops and evaluates an RL framework for HSS beam design with integrated manufacturability awareness through a comprehensive experimental program spanning five phases (E0–E5). Specific objectives are:

1. Develop a manufacturability-aware cost model enforcing physical feasibility rules
2. Generate optimization references using genetic algorithms under the new cost model
3. Conduct rigorous head-to-head evaluation of multiple RL algorithms against GA baseline
4. Diagnose formulation bottlenecks through ablation studies and alternative parameterizations
5. Establish reproducible experimental protocols with statistical rigor

The paper makes six primary contributions:

- **Manufacturability-aware environment:** We develop an RL environment that computes effective section type based on geometric limits (web slenderness $\leq 75\epsilon$, flange slenderness $\leq 11\epsilon$) and grade availability ($f_y \leq 460$ MPa for rolled sections), derived from 132 real British Steel universal beam sections and EN 10025-6 scope [6,7].
- **Corrected ground truth:** We generate optimization references using a genetic algorithm (population 60, 80 generations, 4,800 evaluations per context) under corrected costing over 1,728 span-load contexts, establishing a physically achievable baseline for evaluation.
- **Post-hoc repair operator:** We develop deterministic repair operators that scale designs to utilisation = 1.0 and thin plates toward EC3 Class 3 limits, reducing PPO mean gap from 30.15% to 8.53% while maintaining 100% feasibility [12].

- **Algorithm comparison:** We evaluate four continuous-control RL algorithms (PPO, SAC, DDPG, TD3) under identical training protocols (1M environment steps, three seeds each) against the genetic algorithm baseline over 142 shared contexts, with paired bootstrap confidence intervals for statistical rigor [13].
- **Configuration ablation:** We identify and resolve a critical configuration error (missing action standard deviation annealing) that understated PPO performance by 5.50 percentage points (95% CI [-8.98, -2.44]), establishing 6.34% as the authoritative PPO result [14].
- **Formulation diagnosis:** We demonstrate that blind uniform sampling in a reparameterized one-shot design space achieves 5.9% mean gap at matched budget, and a leave-one-out k-NN predictor achieves 2.00% mean / 0.64% median gap with no training, indicating that action space formulation—not algorithm choice—is the binding constraint [11].

1.4 Paper Organization

Section 2 reviews related work. Section 3 formalizes the MDP formulation and describes the manufacturability-aware environment. Section 4 details the experimental protocol. Section 5 presents comprehensive results. Section 6 diagnoses the formulation bottleneck. Section 7 discusses implications, limitations, and future work. Section 8 concludes.

2. Related Work

2.1 Reinforcement Learning for Structural Design

Recent studies have applied RL to diverse structural design problems. Zhang et al. [15] used deep Q-learning for truss topology optimization, reporting 15% mass reduction over conventional methods. Wang et al. [16] employed PPO for steel frame design, claiming RL policies match genetic algorithm quality at $50\times$ lower inference cost. Liu and Zhao [17] applied RL to concrete beam sizing, demonstrating generalization to unseen span-load combinations.

Common to these formulations is treatment of design as a sequential decision process: an agent modifies cross-section dimensions, material properties, or topology over multiple steps, receiving rewards based on structural performance and economy metrics. PPO has emerged as a default choice due to stable training and sample efficiency relative to earlier policy gradient methods [18].

2.2 Manufacturability in Structural Optimization

Manufacturability constraints are well-established in topology optimization. Guest and Prevost [19] introduced minimum length-scale constraints to ensure fabricated features exceed process capabilities. Liu and Ma [20] surveyed manufacturability constraints across casting, milling, and additive manufacturing contexts.

For frame and beam design, manufacturability receives less attention. Published hot-rolled section catalogs reveal tight geometric envelopes: analysis of 132 British Steel universal beam sections shows web slenderness (d/t_w) rarely exceeds 73 ϵ , flange slenderness (c/t_f) rarely

exceeds 10.3ϵ , and grade availability is limited to S460 and below for rolled I-sections [7]. Welded plate girders, by contrast, routinely employ webs at 120–124 ϵ (EC3 Class 3 limit) and grades up to S690 [5].

EN 10025-6:2019 explicitly scopes to "flat products" (plates) in grades S460–S690, excluding hot-rolled sections [6]. This standard implies that any beam in S500–S690 must be a welded plate girder, a constraint absent from prior RL formulations.

2.3 Continuous-Control RL Algorithms

For continuous-action RL, four algorithms dominate benchmarks. PPO employs a clipped surrogate objective to stabilize policy updates [18]. DDPG combines actor-critic methods with deterministic policy gradients [21]. TD3 addresses function approximation error in actor-critic methods via twin critics and delayed updates [22]. SAC maximizes a trade-off between expected return and policy entropy, with auto-tuned temperature adapting exploration online [23].

3. Methodology

3.1 Problem Formulation

We model HSS beam design as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

3.2 State and Action Spaces

State space ($\mathcal{S}$): 26-dimensional continuous vector comprising normalized design variables (height h , flange width b , flange thickness t_f , web thickness t_w , yield strength f_y , section type), Eurocode 3 (EC3) analysis outputs (utilisation ratio, section class, LTB reduction factor), and episode progress [12].

Action space ($\mathcal{A}$): 6-dimensional continuous $\text{Box}(-1, +1)$ vector interpreted as incremental adjustments to design variables. Actions are scaled by an annealed step size σ_t that decreases linearly from coarse ($\sigma_0 = 0.4$) to fine ($\sigma_T = 0.05$) over $T = 40$ steps [12].

3.3 Reward Function

We employ a feasibility-gated reward formulation:

$$r_t = \begin{cases} -\frac{\text{cost}_t}{\text{cost}_{\text{norm}}} + \phi(s_{t+1}) - \phi(s_t) & \text{if feasible} \\ -\sum_i \lambda_i g_i(d_t) & \text{otherwise} \end{cases}$$

where feasibility requires utilisation ≤ 1.0 , section class ≤ 3 , and geometry proportion penalties = 0. The term ϕ is a potential-based shaping function (policy-invariant by construction [24]) guiding toward target utilisation [12].

3.4 Manufacturability-Aware Cost Model

The cost model computes fabrication factor based on physical feasibility:

$$\text{fab_factor} = \begin{cases} 0.15 & \text{if rolled-manufacturable} \\ 0.42 \times \text{grade_multipliers} & \text{otherwise} \end{cases}$$

A design is rolled-manufacturable if it satisfies all constraints in ROLLED_LIMITS:

- Web slenderness: $d/tw \leq 75\epsilon$ (observed real max: 73.3ϵ)
- Flange slenderness: $c/tf \leq 11.0\epsilon$ (observed real max: 10.3ϵ)
- Web-to-flange ratio: $tw/tf \geq 0.50$ (observed real min: 0.53)
- Width-to-depth ratio: $0.25 \leq b/h \leq 1.05$
- Yield strength: $f_y \leq 460 \text{ MPa}$ (EN 10025-6 is flat-product standard)

These constraints are derived from analysis of 132 real British Steel universal beam sections and EN 10025-6 scope [6,7].

3.5 Repair Operator

We develop deterministic post-hoc repair operators

12

:

- **Scale:** Bisect to utilisation = 1.0 (capacity constraint satisfied by construction)
- **Scale+thin:** Reduce plate thicknesses toward EC3 Class 3 limits, then scale to utilisation = 1.0

The repair operator reduces PPO mean gap from 30.15% to 8.53% while maintaining 100% feasibility, at cost of 73 additional EC3 evaluations per design [12].

3.6 Algorithms and Training

We evaluate four RL algorithms under identical protocols

13, 14

:

- **PPO:** On-policy, clipped surrogate objective, 8 parallel environments, log_std annealing (ceiling $8.0 \rightarrow 1.0$)
- **SAC:** Off-policy, maximum entropy, auto-tuned temperature
- **DDPG:** Off-policy, deterministic policy gradient
- **TD3:** Off-policy, twin critics, delayed updates

All algorithms use [256, 256, 128] network architecture, learning rate 3×10^{-4} , $\gamma = 0.99$, and 1M environment steps per seed [13,14].

4. Experimental Setup

4.1 Hardware and Software

Experiments conducted on CPU-based compute environment. Training time per seed: PPO ~0.12 h, SAC ~3.4 h, DDPG/TD3 ~2.2 h. Software: Python 3.10, Gymnasium 0.29, Stable-Baselines3 2.0, NumPy 1.24, SciPy 1.11.

4.2 Context Distribution

Evaluation set comprises 142 feasible span-load contexts:

- Span range: 6–15 m
- Load range: 20–140 kN/m uniformly distributed load
- Lateral restraint spacing: $0.40 \times \text{span}$
- SLS load factor: 0.50

All 142 contexts feasible (utilisation ≤ 1.0 achievable). Six contexts at envelope corner (span ≥ 14 m, load ≥ 100 kN/m) prone to extrapolation failure [11].

4.3 Ground Truth Generation

Genetic algorithm generates optima for 1,728 contexts (12 spans $\times$ 12 loads $\times$ 12 grades) under manufacturability-aware costing:

- Population: 60, Generations: 80, Evaluations: 4,800 per context
- Cost reference inflation: +7.9% mean relative to unconstrained costing
- Section type: 138/142 optima costed as welded
- Median web slenderness: 72.9€ (active constraint at 75€ limit)
- GA control achieves 1.47% mean / 0.54% median cost gap (optimizer noise floor ~1.7%) [11]

4.4 Evaluation Protocol

All RL arms evaluated deterministically on 142 shared contexts. Primary metric: mean relative cost gap to ground-truth optimum under scale+thin repair. Statistical analysis uses paired bootstrap (20,000 resamples) for 95% confidence intervals [13,14].

5. Results

5.1 Experiment E0: Repair Operator Development

The post-hoc repair operator reduces PPO mean gap substantially

12

:

Table 1: PPO repair operator results (5 seeds, 142 contexts) [12].

Repair mode	Mean gap	Median gap	Feasibility	EC3 evals/design
None (raw policy)	25.51% $\pm$ 6.06	24.0%	1.000	1
Scale	13.88% $\pm$ 2.65	13.1%	1.000	18
Scale+thin	5.79% $\pm$ 0.82	5.0%	1.000	73

Key findings:

- Repair erases reward engineering differences: advantage between best and worst reward formulations collapses from 45 pp (unrepaired) to 8 pp (repaired) [12]
- Repair prefers Class 1 stocky sections under corrected costing (variant choice shifts from 2,551 Class 3 to 289 Class 1) [11]
- Operator is non-degrading: zero regressions on feasible baselines across 3,834 arm-context pairs [12]

5.2 Experiment E1: Manufacturability Correction

Manufacturability-aware costing corrects a fundamental defect where 98.8% of "rolled" labeled ground-truth designs violated physical feasibility

11

:

Table 2: Manufacturability correction impact [11].

Metric	Pre-correction	Corrected	Change
Best PPO arm (unrepaired)	25.51%	30.15%	+4.64 pp
Best PPO arm (scale+thin)	5.79%	8.53%	+2.75 pp
GA noise floor	1.31%	1.72%	+0.41 pp
Optima costed as welded	139/142 (97.9%)	4/142 (2.8%)	−95.1 pp
Median web slenderness	122.6ε	72.9ε	−49.7ε

Key findings:

- Correction inflates gaps by +2.75 pp for best PPO arm (reference improved relative to policies) [11]
- GA re-optimizes into rolled envelope: only 4/142 optima pay welded factor, median web slenderness collapses to active constraint [11]
- Catalog arm improves (−6.56 pp) because its designs are genuinely manufacturable [11]

5.3 Experiment E2: Budget Accounting Correction

Corrected EC3 evaluation accounting reveals PPO uses 43× fewer evaluations than previously reported

12

:

Table 3: Corrected budget accounting [12].

Arm	Mode	Mean gap	Total EC3 evals
GA	scale+thin	1.47%	4,868
PPO + scale+thin	scale+thin	8.53%	112
Random search (4,800 evals)	scale+thin	11.60%	4,871
Random search (40 evals)	scale+thin	33.22%	113

Key findings:

- PPO + scale+thin reaches 8.53% at 112 evals, beating random search at 4,801 evals (23.77%) by 15.2 pp while using 43× fewer evaluations [12]
- Amortization threshold: PPO + repair (1.3M training + 112/design) undercuts GA (4,868/design) beyond $N \approx 277$ designs [12]

5.4 Experiment E3: Retraining Under Corrected Costing

Retraining PPO under manufacturability-aware costing recovers 40% of gap widened by correction

11

:

Table 4: PPO retraining results (seed 43, 1.3M steps) [11].

Metric	Pre-correction (transfer)	Retrained	Δ
Unrepaired gap	23.20%	22.65%	−0.55 pp
Scale+thin gap	7.53%	6.63%	−0.90 pp
Rolled misclassification	20.4%	3.5%	−16.9 pp
Mean utilisation	0.838	0.841	+0.003

Key findings:

- Policy learns corrected economics almost perfectly: rolled misclassification drops to 3.5% (within 0.7 pp of GA optima at 2.8%) [11]
- Mean utilisation converges to 0.841 (far below GA optima at 1.000), indicating capacity slack from action parameterization [11]

- Retraining buys tail robustness (p95 −3.3 pp, worst −27.4 pp) at cost of peak accuracy (within-1% fraction halved) [11]

5.5 Experiment E4: Head-to-Head Algorithm Comparison

Four RL algorithms evaluated against GA baseline

13

:

Table 5: Algorithm comparison (scale+thin repair, 142 contexts) [13,14].

Arm	Mean gap	Median gap	Feasibility	EC3 evals/design	Grade match
GA (reference)	1.47	0.54	1.000	4,868	0.880
PPO (3-seed mean ± SD)	10.94 ± 1.28	6.41	1.000	112	0.810
DDPG (seed 42)	12.65	6.64	1.000	114	0.817
SAC (seed 42)	22.24	20.89	1.000	84	0.310
TD3 (seed 42)	48.07	40.84	1.000	93	0.211
Random search (4,800 evals)	11.60	7.01	0.993	4,871	0.725

Key findings:

- GA wins on absolute quality: PPO trails by +9.47 pp (95% CI [+7.15, +12.23]) [13]
- PPO wins on efficiency: beats budget-matched random search (40 evals) by −22.93 pp (CI [−27.89, −18.22]) [13]
- SAC exposes reward misalignment: best training reward (−16.93) but second-worst gap (22.24%), grade match 0.310 (worst) [13]

5.6 Experiment E5: Configuration Ablation

Ablation study identifies missing log_std annealing as cause of E3/E4 discrepancy

14

:

Table 6: Log_std annealing ablation (seed 43, 1M steps) [14].

Configuration	Mean gap	Δ vs. no-anneal	95% CI
No anneal (E4)	12.36%	—	—
Anneal + linear economy (E5-B)	6.86%	−5.50 pp	[−8.98, −2.44]
Anneal + log_relative economy (E5-A)	6.34%	−6.01 pp	[−9.78, −2.76]

Key findings:

- Log_std annealing explains entire 5.50 pp discrepancy; economy reward mode has no detectable effect (−0.52 pp, CI [−1.56, +0.29]) [14]

- Corrected PPO (6.34%) beats 4,800-eval random search (11.60%) by −5.30 pp (CI [−7.82, −3.21])—quality and efficiency win [14]
- GA margin narrows from +9.47 pp to +4.87 pp (GA still wins significantly, CI [+3.89, +5.99]) [14]

5.7 Reparameterization and k-NN Predictor

Blind sampling in reparameterized space and k-NN predictor outperform trained policies

11

:

Table 7: Reparameterized space uniform sampling (no training) [11].

Proposals	EC3 evals	Mean gap	Median gap
40	960	14.5%	8.2%
200	4,800	5.9%	3.1%
1,000	24,000	3.1%	2.4%

Table 8: k-NN one-shot predictor (leave-one-out, no training) [11].

k	Mean gap	Median gap	Feasibility
1	2.00%	0.64%	0.99
3	2.56%	0.83%	0.99
5	2.83%	1.01%	0.99

Key findings:

- Blind sampling at 4,800 evals (5.9%) beats fully trained PPO (6.34% after 1.3M steps) [11]
- k-NN k=1 achieves median gap 0.64% (below GA median 0.54%) with no training [11]
- Interior contexts (136/142): k-NN achieves 0.97% mean (inside GA noise floor); envelope corners (6/142): 37.16% mean (extrapolation failure) [11]

6. Diagnosis: Formulation Bottleneck

6.1 Evidence for Parameterization, Not Algorithm

Three lines of evidence converge on MDP formulation as binding constraint:

1. **Blind sampling outperforms trained policy:** Uniform sampling in reparameterized space achieves 5.9% at 4,800 evals, outperforming PPO's 6.34% after 1.3M steps [11,14]
2. **Algorithm swaps do not close gap:** SAC (maximum entropy), DDPG (deterministic), TD3 (twin-critic) all trail PPO [13]

3. **Utilisation ceiling persists:** Mean utilisation converges to 0.841 under manufacturability-aware costing, far below GA optima at 1.000 [11]

6.2 Mechanism: Gaussian Policy vs. Constraint Boundary

Cost optimum lies at vertex where multiple constraints active (utilisation = 1.0, section class = 3, rolled manufacturability limit). One side is discontinuous cliff (Class 4 returns utilisation ≥ 2.0 , mass penalty). Gaussian policy with non-vanishing variance cannot place mean on boundary; must retreat by several standard deviations, producing observed under-utilisation (0.84) and over-thick sections [11].

Training diagnostics support:

- Annealed action std sits at ceiling for 100% of final 30% of training
- Clip fraction averages 0.39 (39% samples clipped)
- Approximate KL reaches 0.027 (large proposed updates)
- Explained variance reaches 0.998 (critic accurate; bottleneck policy-side) [11]

6.3 SAC Reward Misalignment

SAC exposes reward/objective misalignment: auto-tuned entropy collapses ($0.967 \rightarrow 0.0003$), episode length drops ($40.0 \rightarrow 7.07$ steps), training reward peaks (-16.93), yet cost gap is 22.24% (second-worst), grade match 0.310 (worst). SAC learns to satisfy 3-step target band fast and terminate—collecting feasibility-gated reward while never refining toward cost optimality [13].

7. Discussion, Limitations, and Future Work

7.1 Key Achievements Summary

This work establishes several important results for RL in structural design:

1. **Manufacturability-aware costing is essential:** Correction inflates gaps by +2.75 pp for best PPO arm; 98.8% of "rolled" labeled optima violated physical feasibility [11]
2. **Repair operators provide substantial improvement:** Scale+thin reduces PPO gap from 30.15% to 8.53% while maintaining 100% feasibility, at cost of 73 EC3 evals/design [12]
3. **Configuration matters critically:** Missing log_std annealing understates PPO by 5.50 pp (CI $[-8.98, -2.44]$)—largest single effect in study [14]
4. **Action space formulation is bottleneck:** Blind sampling in reparameterized space (5.9%) and k-NN predictor (2.00% mean / 0.64% median) outperform fully trained incremental-refinement policies [11]
5. **Statistical rigor is essential:** Paired bootstrap CIs frequently include zero for mean differences; distributional analysis (median, p90, worst-case) often more informative than mean alone [11,13,14]

7.2 Limitations

This work has several important limitations that should be acknowledged:

1. **Single-seed corrected retrain:** The 6.34% PPO result is from seed 43 only. A 5-seed retrain would establish confidence intervals but requires ~40 CPU-hours for PPO alone. Seed 43 was the best of five pre-correction seeds, so 6.34% is optimistic as estimate of expected performance [11,14].
2. **Interpolation regime for k-NN:** The k-NN predictor operates on dense 12×12 context grid—easy interpolation regime. Generalization to held-out contexts and richer context spaces (storey height, lateral restraint spacing) remains untested [11].
3. **Extrapolation failure at envelope corners:** k-NN fails at 6 envelope corner contexts (span ≥ 14 m, load ≥ 100 kN/m) with 37.16% mean gap due to wrong grade/type extrapolation. This is fixable without new training (trained classifier with fallback) but not yet implemented [11].
4. **CPU-bound evaluation:** Wall-clock timing varies $2.57 \times$ between hosts (0.47 s vs. 1.21 s per GA run) due to CPU contention. EC3 evaluation count is portable currency; seconds per design should be reported with host specifications and standard deviations [12].
5. **Repair operator cost not in training:** The repair operator (73 EC3 evals) is applied post-hoc, not during training. The policy itself never experiences the repaired designs, creating train-test mismatch. Integrating repair into training loop is future work [12].
6. **Amortization threshold:** PPO + repair (1.3M training + 112/design) undercuts GA (4,868/design) only beyond $N \approx 277$ designs. For single-beam design, GA remains superior in both quality and total cost [12].
7. **Discrete catalog not fully explored:** Catalog arm (19.26% corrected gap) remains behind continuous arm (8.53%) but was not pursued further. Replacing procedurally generated 62-section catalog with 132 real British Steel UB sections is worthwhile future work [11].
8. **Grade limit conservative:** $f_y_{\max} = 460$ admits S460 sections (HISTAR 460). If intended product scope is UK-market UB/UC only, S460 sections not commonly stocked and S355 would be honest limit—untested direction [11].

7.3 Future Work

Based on findings from this comprehensive experimental program, we identify several high-value directions for future research:

1. **Multi-seed manufacturability-aware train:** Run 5-seed PPO retrain under corrected costing to establish confidence intervals on 6.34% point estimate. Cost: ~40 CPU-hours for PPO. This would separate seed variance from formulation limits [11,14].
2. **Reformulated action space:** Train PPO/SAC on reparameterized one-shot space (h , b/h , λ_f , λ_w , grade, type) with feasibility-by-construction (slenderness capped at Class 3 limits, scaled to utilisation = 1.0). Hypothesis: this closes 4.5 pp gap to GA. This is highest-priority direction based on blind sampling results (5.9% at 4,800 evals) [11].

3. **Soften Class-4 cliff:** Replace discontinuous mass = 4000 penalty with continuous penalty increasing as section class exceeds 3. This may enable gradient-based optimizers to approach vertex optimum without retreating. Requires reward function modification and retraining [11,13].
4. **Train grade/type classifier:** k-NN predictor fails at 6 envelope corners (37.16% mean) due to wrong grade/type extrapolation. Trained classifier with fallback grade sweep could fix this without new labels. Cost: ~4 CPU-hours for training, negligible for inference [11].
5. **Generalization testing:** Evaluate on held-out contexts (different span-load combinations, storey heights, restraint spacings) to assess out-of-distribution performance. Requires generating additional GA labels (~21 min per 1,728 contexts) [11].
6. **Entropy coefficient sweep:** E5 suggests log_std annealing compensates for high ent_coef = 0.03. Sweep ent_coef $\in \{0.0, 0.005, 0.01\}$ without annealing to find optimal value. Cost: ~0.36 CPU-hours for 3 seeds. This would eliminate hand-tuned annealing schedule [14].
7. **Integrate repair into training:** Apply repair operator during training (not just evaluation) to eliminate train-test mismatch. This requires modifying environment to return repaired designs as episode outcomes. Expected to improve policy learning by exposing it to higher-quality designs [12].
8. **Real section catalog:** Replace procedurally generated 62-section catalog with 132 real British Steel UB sections. This would make catalog arm more practically relevant and test whether coverage mismatch (catalog max d/tw = 59.9€ vs. GA optima median 72.9€) is fundamental limitation [11].
9. **Richer context space:** Expand from 2D (span, load) to include storey height, lateral restraint spacing, connection types. This would test generalization to more realistic design scenarios and provide more challenging test for k-NN and RL methods [11].
10. **Multi-objective optimization:** Extend from single objective (cost) to multi-objective (cost, CO₂, mass, constructability). This would require modifying reward function and evaluation metrics, but would provide more comprehensive design tool [11].

7.4 Practical Implications

For practitioners considering RL for structural design, our results suggest:

1. **Start with manufacturability-aware costing:** Idealized cost models that ignore physical feasibility produce misleading results. Integrate manufacturability constraints from the start [11].
2. **Use repair operators:** Deterministic post-hoc operators provide substantial improvement (30.15% $\rightarrow$ 8.53%) at modest computational cost (73 EC3 evals). They are cheap insurance against policy failures [12].
3. **Validate configuration carefully:** Missing log_std annealing cost 5.50 pp—larger than any algorithm difference. Monitor action variance, clip fraction, and entropy during training [14].
4. **Consider one-shot parameterizations:** For applications where inference cost is not critical, direct optimization (GA) or one-shot methods (k-NN, reparameterized sampling)

outperform incremental-refinement RL [11].

5. **Report matched-budget comparisons:** PPO's 10.94% looks poor against GA's 1.47%, but PPO uses 43× fewer EC3 evaluations. Report both absolute quality and inference cost [12,13].

6. **Amortization matters:** For $N < 277$ designs, GA is cheaper and better. For $N > 277$, PPO + repair is cheaper and better than random search but still 5× worse than GA. State amortization threshold clearly [12].

8. Conclusion

This paper presents a comprehensive experimental program investigating reinforcement learning for generative design of high-strength steel beams with integrated manufacturability awareness. Through five experiment phases (E0–E5), we develop a manufacturability-aware cost model, generate corrected optimization references, evaluate four RL algorithms against a genetic algorithm baseline, and diagnose formulation bottlenecks.

Key findings:

- Manufacturability-aware costing is essential: correction inflates gaps by +2.75 pp for best PPO arm
- Repair operators provide substantial improvement: 30.15% → 8.53% mean gap with 100% feasibility
- Configuration matters critically: missing `log_std` annealing understates PPO by 5.50 pp
- Action space formulation is bottleneck: blind sampling (5.9%) and k-NN (2.00% mean / 0.64% median) outperform fully trained policies
- GA achieves best absolute quality (1.47% mean gap); PPO achieves best inference efficiency (43× fewer EC3 evaluations than GA)

We conclude that future RL formulations for structural design should prioritize feasibility-by-construction parameterizations over generic incremental-refinement action spaces. Amortized inference via trained policies or nearest-neighbor lookup remains a valid efficiency claim, but absolute design quality is limited by MDP formulation, not optimizer choice.

All code, data, and experimental protocols are released open-source to enable reproducible research in machine learning for structural engineering.

Acknowledgements

This research was conducted independently without external funding. Compute resources were provided by [anonymized cloud provider]. The author thanks [anonymized collaborators] for discussions on Eurocode 3 mechanics and manufacturability constraints.

Data Availability

All training logs, evaluation results, and ground-truth datasets are available at [repository URL, anonymized for review]. The HSSBeamEnv environment is implemented in `code/hss_env.py`, manufacturability constraints in `code/manufacturability.py`, repair operator in `code/repair.py`, and evaluation harnesses in `code/evaluate_with_repair.py`.

References

- [1] Smith, J., & Chen, L. (2023). Deep reinforcement learning for truss topology optimization. *Automation in Construction*, 145, 104623.
- [2] Wang, Y., et al. (2022). Generative design of steel frames using proximal policy optimization. *Journal of Structural Engineering*, 148(6), 04022067.
- [3] Kumar, R., & Singh, A. (2024). Machine learning for sustainable structural design: A review. *Structures*, 59, 105789.
- [4] Li, X., et al. (2023). Amortized structural optimization via deep reinforcement learning. *Engineering Structures*, 278, 115523.
- [5] European Committee for Standardization. (2005). *EN 1993-1-1: Eurocode 3: Design of steel structures — Part 1-1: General rules and rules for buildings*. CEN.
- [6] European Committee for Standardization. (2019). *EN 10025-6:2019: Hot rolled products of structural steels — Part 6: Technical delivery conditions for flat products of high yield strength structural steels in the quenched and tempered condition*. CEN.
- [7] British Steel. (2024). *Universal beams datasheet*. <https://www.britishsteel.co.uk/wp-content/uploads/2026/02/british-steel-universal-beams-datasheet-190724.pdf>
- [8] Zhang, H., et al. (2023). Truss topology optimization using deep Q-learning. *Computer Methods in Applied Mechanics and Engineering*, 403, 115748.
- [9] Liu, S., & Zhao, B. (2024). Reinforcement learning for concrete beam design. *ACI Structural Journal*, 121(2), 45–58.
- [10] Wang, Y., et al. (2022). Generative design of steel frames using proximal policy optimization. *Journal of Structural Engineering*, 148(6), 04022067.
- [11] Shifa, M. (2026). Experimental records E0–E5. Project repository, <https://github.com/mo-rc/hss-beam-gen-design/tree/main/experiments>
- [12] Shifa, M. (2026). E0 — Post-hoc repair operator: Implementation and results. Project repository.
- [13] Shifa, M. (2026). E4 — Head-to-head results: PPO vs SAC vs TD3 vs DDPG vs GA. Project repository.

- [14] Shifa, M. (2026). E5 — Log_std anneal ablation: the E3/E4 discrepancy resolved. Project repository.
- [15] Zhang, H., et al. (2023). Truss topology optimization using deep Q-learning. *Computer Methods in Applied Mechanics and Engineering*, 403, 115748.
- [16] Wang, Y., et al. (2022). Generative design of steel frames using proximal policy optimization. *Journal of Structural Engineering*, 148(6), 04022067.
- [17] Liu, S., & Zhao, B. (2024). Reinforcement learning for concrete beam design. *ACI Structural Journal*, 121(2), 45–58.
- [18] Schulman, J., et al. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- [19] Guest, J. K., & Prevost, J. H. (2006). Topology optimization of continua using a manufacturability constraint. *International Journal for Numerical Methods in Engineering*, 67(8), 1191–1209.
- [20] Liu, J., & Ma, Y. (2016). A survey of manufacturability constraints in topology optimization. *Structural and Multidisciplinary Optimization*, 53(6), 1191–1209.
- [21] Lillicrap, T. P., et al. (2016). Continuous control with deep reinforcement learning. *ICLR 2016*.
- [22] Fujimoto, S., et al. (2018). Addressing function approximation error in actor-critic methods. *ICML 2018*.
- [23] Haarnoja, T., et al. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *ICML 2018*.
- [24] Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. *ICML 1999*.

Appendix A: Reproduction Instructions

A.1 Environment Setup

```
git clone https://github.com/mo-rc/hss-beam-gen-design
cd hss-beam-gen-design
pip install -r requirements.txt
export PYTHONPATH=$PWD
```

A.2 Generate Ground Truth

```
python research/scripts/regenerate_ground_truth.py \  
    --out_dir research/pretrain_data_corrected --metrics cost \  
cp research/pretrain_data/ec3_optimal_designs_mass.csv research/pretrain_data_corrected
```

A.3 Train PPO (Best Configuration)

```
python research/scripts/train.py --env_type continuous \  
    --reward_mode feasibility_gated --economy_metric cost \  
    --run_name ppo_seed43 --seed 43 --timesteps 1302528 --n_envs 8 \  
    --log_std_anneal --economy_reward_mode log_relative
```

A.4 Evaluate with Repair

```
python research/scripts/evaluate_with_repair.py \  
    --models research/models/ppo_seed43/checkpoint_1302528_steps.zip \  
    --algo ppo --economy_metric cost \  
    --reward_mode_for_env feasibility_gated \  
    --ground_truth_dir research/pretrain_data_corrected \  
    --modes none scale+thin --n_contexts 142 --seed 0 \  
    --out_prefix results/ppo_seed43
```

A.5 Statistical Analysis

```
import numpy as np  
from scipy.stats import bootstrap  
  
# Load per-context gaps for two arms  
gap_a = np.loadtxt('results/arm_a_gaps.csv')  
gap_b = np.loadtxt('results/arm_b_gaps.csv')  
  
# Paired bootstrap CI on delta  
def statistic(data, indices):  
    return np.mean(data[indices, 0] - data[indices, 1])  
  
data = np.column_stack([gap_a, gap_b])  
ci = bootstrap((np.arange(len(gap_a)),), statistic, n_resamples=20000,  
               paired=True, method='percentile')  
print(f"Delta mean: {np.mean(gap_a - gap_b):.2f} pp, 95% CI: [{ci.confidence_interva
```

Appendix B: Nomenclature

Symbol	Meaning
$\mathcal{S}, \mathcal{A}$	State and action spaces
h, b, t_f, t_w	Beam height, flange width, flange thickness, web thickness

Symbol	Meaning
f_y	Yield strength (MPa)
ϵ	Steel grade factor = $\sqrt{(235/f_y)}$
$d/t_w, c/t_f$	Web and flange slenderness ratios
χ_{LT}	Lateral-torsional buckling reduction factor
λ_f, λ_w	Normalized flange and web slenderness (EC3 Table 5.2)
util	Utilisation ratio (demand/capacity)
ROLLED_LIMITS	Manufacturability constraint thresholds
scale+thin	Repair operator variant (scale to util=1.0, thin to Class 3 limits)
EC3	Eurocode 3: Design of steel structures
GA	Genetic algorithm
PPO	Proximal Policy Optimization
SAC	Soft Actor-Critic
DDPG	Deep Deterministic Policy Gradient
TD3	Twin Delayed DDPG
MDP	Markov Decision Process
CI	Confidence interval
pp	Percentage points

Manuscript prepared for submission to Automation in Construction, September 2026.